

Project 1



UTM

UNIVERSITI TEKNOLOGI MALAYSIA

FACULTY OF COMPUTING

SEMESTER II

2024/2025

SECP 2753 – DATA MINING

SECTION 01

PROJECT 1 - SUPERVISED LEARNING

LECTURER: DR. ROLIANA BINTI IBRAHIM

NAME	MATRIC NUMBER
NURUL IKA SYAFINY BINTI AZHAR	A23CS0164
LUBNA AL HAANI BINTI RADZUAN	A23CS0107
NURAI SYAH BINTI MOHD ZIKRE	A23CS0160
NUR FIRZANA BINTI BADRUS HISHAM	A23CS0156

Table of Contents

1.0 Introduction	2
2.0 Introduction to the Dataset: No Show Appointment	3
3.0 Business and Data Understanding	3
4.0 Data Preparation and Preprocessing	4
5.0 Model Development (Supervised Learning)	6
5.1 Model 1: Random Forest Classifier	7
5.2 Model 2: Logistic Regression	7
6.0 Model Performance & Evaluation (Accuracy, Precision, Recall, F1)	10
6.1 Model 1: Random Forest Classifier	10
6.2 Model 2: Logistic Regression	11
7.0 Conclusion	14

1.0 Introduction

In today's data-driven world, organizations and companies are constantly seeking for insights from their data to make strategic decisions and optimize operations. In this project, we apply 2 different supervised machine learning techniques onto a dataset: appointment no-shows dataset. By using 2 different predictive models with a dataset, we aim to compare the performances and the accuracy of the models used.

In this project, both descriptive and predictive analytics will be applied. Exploratory data analysis (EDA) will be conducted to understand feature distributions, correlations, and trends related to no-show to appointment behavior. Following that, two machine learning classification models, **Random Forest and Logistic Regression**, will be trained and evaluated. The goal is to identify key factors contributing to missed appointments and to build a predictive model that can help healthcare providers anticipate and reduce no-shows.

Both dataset will go through a comprehensive process of business and data understanding, preparation, model development, and performance evaluation. We will utilize key metrics, such as, **accuracy, precision, recall and F1-score** to demonstrate the effectiveness of supervised machine learning techniques in gaining insights from complex datasets. Ultimately, we will do a comparison of key metrics between the random forest model and the logistic regression model.

2.0 Introduction to the Dataset: No Show Appointment

In this report, we are going to analyze the dataset regarding patients who missed their medical appointments as well as running a machine learning model to predict whether the patient will likely show up to their medical appointment or not and evaluate its performance. The original dataset can be obtained from this link: [📄 noshowappointments](#)

3.0 Business and Data Understanding

Problem Statement:

To predict whether a patient will likely show up for their scheduled medical appointment.

Overview:

In this part, we will focus on understanding the business problem and exploring the dataset where the main goal is to predict whether a patient will likely show up for their scheduled medical appointment, to improve the management in the healthcare ecosystem. Firstly, we will explore the dataset structure, types of features, and initial data quality. Basic analysis helps identify key characteristics such as missing values, imbalanced classes, and anomalies (e.g., negative age). These insights guide the data cleaning and feature engineering steps that follow.

Importance:

- By identifying a patient that has high risk of missing their medical appointment, an alternative action can be taken by reminding them of their medical appointment and rescheduling it if any.
- Predicting a potential no-shows trends can help healthcare providers in managing the schedule by rescheduled the unused appointment to reduce downtime

4.0 Data Preparation and Preprocessing

This part of the report plays a crucial role in machine learning that focuses on cleaning, transforming, and structuring raw data for effective analysis and model training. Its importance lies in ensuring data quality to enable robust feature engineering, facilitating necessary data transformations and improving model performance and prediction accuracy.

Overview of Data Preparation and Its Importance

- **Handling Data Quality Issues:** Addressing anomalies like negative age or inconsistent date formats to prevent biased models.
- **Feature Engineering:** Creating new, more informative features (e.g., WaitingDays) from existing ones to enhance predictive power.
- **Data Transformation:** Converting categorical data (e.g., 'Gender', 'No-show', 'Neighbourhood') into numerical formats required by machine learning algorithms.
- **Dimensionality Management:** Using one-hot encoding for categorical features while being mindful of potential dimensionality increases.
- **Improving Model Performance:** Ensuring clean, well-structured data for accurate pattern learning and better generalization.

Steps to Prepare Data for Supervised Learning

1. **Loading and Initial Inspection:** Loading no_show_appointment.csv and checking its structure and initial value distributions.
2. **Data Cleaning:** Removing entries with negative 'Age' values for data consistency.
3. **Date Conversion:** Converting 'ScheduledDay' and 'AppointmentDay' to datetime objects for calculations.
4. **Feature Engineering:**
 - Calculating WaitingDays (days between scheduling and appointment) and ensuring non-negative values.
 - Extracting AppointmentDayOfWeek, ScheduledDayOfWeek, and MonthOfAppointment to capture temporal patterns.
5. **Categorical Encoding:**

- Mapping 'No-show' ('No' to 0, 'Yes' to 1) and 'Gender' ('F' to 0, 'M' to 1) to numerical values.
- 6. **Column Dropping:** Removing non-predictive columns (PatientId, AppointmentID, original date columns).
- 7. **One-Hot Encoding:** Applying one-hot encoding to the 'Neighbourhood' column to convert categorical locations into numerical features.

Application of Data Preprocessing Methods

The data preprocessing steps were implemented using Python's pandas and sklearn libraries. The code transforms the raw dataset into a clean and prepared format suitable for model training. A full code snippet to the process for this phase is shown below.

```
# Handle anomaly in Age
df = df[df['Age'] >= 0]

# Convert to datetime
df['ScheduledDay'] = pd.to_datetime(df['ScheduledDay'])
df['AppointmentDay'] = pd.to_datetime(df['AppointmentDay'])

# Feature: WaitingDays
df['WaitingDays'] = (df['AppointmentDay'] - df['ScheduledDay']).dt.days
df['WaitingDays'] = df['WaitingDays'].apply(lambda x: x if x >= 0 else 0)

# Date components
df['AppointmentDayOfWeek'] = df['AppointmentDay'].dt.dayofweek
df['ScheduledDayOfWeek'] = df['ScheduledDay'].dt.dayofweek
df['MonthOfAppointment'] = df['AppointmentDay'].dt.month

# Encode categorical columns
df['No-show'] = df['No-show'].map({'No': 0, 'Yes': 1})
df['Gender'] = df['Gender'].map({'F': 0, 'M': 1})

# Drop unnecessary columns
df_processed = df.drop(columns=['PatientId', 'AppointmentID', 'ScheduledDay', 'AppointmentDay'], axis=1)

# Remove negative waiting days
df = df[df['WaitingDays'] >= 0]

print("\n---First 5 Rows of Preprocessed Data---")
df_processed.head()

print("\n--DataFrame info before one-hot encoding 'Neighborhood'--")
print(df_processed.info())
df[['Neighbourhood', 'Gender', 'Age', 'No-show']].head(3)

# Apply One-Hot Encoding to 'Neighbourhood' column
print("\nDataFrame info after one-hot encoding 'Neighborhood'")
df_prep = pd.get_dummies(df_processed, columns=['Neighbourhood'], prefix='Neighbourhood')
print(df_prep.info())
```

5.0 Model Development (Supervised Learning)

Supervised Learning is a fundamental machine learning approach where an algorithm learns from a labeled dataset. It means that during the training phase, the algorithm is provided with input data points, each associated with a corresponding correct output label. The objective of the algorithm is to discover a mapping function from these input features to their respective output labels, enabling it to accurately predict labels for new and unseen data. For this project, to do supervised learning, we used python to analyze it.

The process involves:

- Input Features (X): These are the independent variables or attributes of the data (e.g., patient's age, gender, appointment details, medical conditions).
- Output Label (y): This is the dependent variable or target that the algorithm aims to predict (e.g., whether a patient 'No-showed' or 'Shown Up').

```
# Split dataset into training and testing (80/20 split)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

- `X_train, X_test, y_train, y_test = train_test_split(...)`: Splits the data into training and testing sets using scikit-learn's `train_test_split` function.
- `test_size = 0.2`: Allocates 20% of the data for testing and 80% for training.
- `random_state = 42`: Sets a seed for reproducible results.
- `stratify=y`: Ensures that the train and test sets have approximately the same proportion of target class values as the original dataset, which is important for imbalanced classification problems.

```
# Starting Random Forest: Initialize Random Forest
rf_model = RandomForestClassifier(n_estimators=200, random_state=42, class_weight='balanced')
print(rf_model.n_estimators)
print(rf_model.class_weight)
```


5.1 Model 1: Random Forest Classifier

The **Random Forest Classifier** is an ensemble learning method that combines predictions from many individual decision trees to achieve robust and accurate results. The basic concept is each decision tree is trained on a random subset of the data and considers only a random subset of features for each decision. For a new prediction, all trees vote and the majority vote determines the final classification.

The Random Forest Classifier was trained using the X_train and y_train data, configured with n_estimators=200 and class_weight='balanced' to manage the class imbalance. After training, the model's feature importances was extracted to identify the most influential factors.

```
--- Top 10 Most Important Features for Predicting No-Shows ---
Age                                0.260576
WaitingDays                        0.220535
ScheduledDayOfWeek                 0.072607
AppointmentDayOfWeek               0.067957
Gender                             0.030226
MonthOfAppointment                 0.023353
SMS_received                       0.018582
Hypertension                       0.010991
Scholarship                        0.010505
Neighbourhood_JARDIM CAMBURI      0.008562
```

As observed, **Age** is the biggest reason why patients miss their appointments. This could be because people of different ages have different levels of responsibility, health knowledge or having problems getting to the clinic. **WaitingDays**, which is the duration between scheduling and the actual appointment is the second most important feature. A longer waiting period can increase the chances of a no-show due to forgotten appointments, changing circumstances or finding alternative care.

5.2 Model 2: Logistic Regression

This section introduces Logistic Regression as a second supervised learning model to predict patient no-shows. Logistic Regression is a statistical model used for binary classification tasks, predicting the probability of an outcome (e.g., show or no-show) based on one or more independent variables.

```
[32]: # Import Logistic Regression
      from sklearn.linear_model import LogisticRegression

      # Initialize and Train the Logistic Regression Model
      # Using a 'liblinear' solver for binary classification and 'lbfgs' is also a good default.
      # max_iter can be increased if convergence warnings appear.

      print("Starting Logistic Regression model training...")
      log_reg_model = LogisticRegression(solver='liblinear', random_state=42, max_iter=1000)
      log_reg_model.fit(X_train, y_train)
      print("Logistic Regression model training completed.")

      # Make predictions on the test set
      y_pred_log_reg = log_reg_model.predict(X_test)
      y_proba_log_reg = log_reg_model.predict_proba(X_test)[:, 1] # Probability of the positive class

      print("\nShape of predictions (y_pred_log_reg):", y_pred_log_reg.shape)
      print("First 10 binary predictions (y_pred_log_reg):")
      print(y_pred_log_reg[:10])

      print("\nShape of probabilities (y_proba_log_reg):", y_proba_log_reg.shape)
      print("First 10 probabilities for the positive class (y_proba_log_reg):")
      print(y_proba_log_reg[:10])
```

This code snippet is dedicated to implementing and getting initial insights from a Logistic Regression model.

`sklearn.linear_model import LogisticRegression` imports the `LogisticRegression` class from scikit-learn's linear models module. This class provides the functionality to build and train a Logistic Regression model.

`log_reg_model = LogisticRegression(...)` creates a Logistic Regression model using the 'liblinear' solver, which is suitable for small, binary classification tasks. The `random_state=42` ensures consistent results across runs, and `max_iter=1000` sets the maximum number of training cycles. The model is then trained using `log_reg_model.fit(X_train, y_train)`, where `X_train` contains the input features and `y_train` holds the target labels (e.g., whether a patient showed up).

After training the model, `y_pred_log_reg = log_reg_model.predict(X_test)` generates binary predictions (like 0 for "show" and 1 for "no-show") using unseen test data. Then, `y_proba_log_reg = log_reg_model.predict_proba(X_test)[:, 1]` calculates the probability of each sample belonging to the positive class (e.g., "no-show"). This is helpful for further evaluation like plotting ROC curves, where probability scores are needed instead of just binary outcomes. Below is the outcome of the model training process.

```
Starting Logistic Regression model training...
Logistic Regression model training completed.

Shape of predictions (y_pred_log_reg): (22106,)
First 10 binary predictions (y_pred_log_reg):
[0 0 0 0 0 0 0 0 0 0]

Shape of probabilities (y_proba_log_reg): (22106,)
First 10 probabilities for the positive class (y_proba_log_reg):
[0.33551792 0.09632019 0.28423053 0.17931871 0.16588985 0.16776603
 0.19414265 0.14651866 0.49140487 0.12634692]
```

“Starting Logistic Regression model training...” indicate when the model training process begins. “Logistic Regression model training completed.” confirms that the training process has successfully finished. “Shape of predictions, First 10 binary predictions, Shape of probabilities, First 10 probabilities” are included to provide immediate feedback on the format and initial values of the predictions and probabilities.

6.0 Model Performance & Evaluation (Accuracy, Precision, Recall, F1)

Evaluating the model on unseen test data is important. Given the dataset's imbalance, multiple metrics are used by simple accuracy.

6.1 Model 1: Random Forest Classifier

Application of Suitable Evaluation Metrics:

```
--- Model Evaluation Results ---
```

```
Confusion Matrix :  
[[16409  1233]  
 [ 3634   830]]
```

- Confusion Matrix:

- True Negatives (TN): **16,409** patients who actually showed up were correctly predicted to show up.
- False Positives (FP): **1,233** patients who actually showed up were incorrectly predicted to no-show.
- False Negatives (FN): **3,634** patients who actually no-showed were incorrectly predicted to show up.
- True Positives (TP): **830** patients who actually no-showed were correctly predicted to no-show.

```
--- Classification Report ---  
              precision    recall  f1-score   support  
  
Showed Up (0)       0.82      0.93      0.87      17642  
No-Show (1)         0.40      0.19      0.25       4464  
  
   accuracy              0.78      22106  
  macro avg              0.61      0.56      0.56      22106  
weighted avg              0.73      0.78      0.75      22106
```

The analysis also focuses on interpreting the metric, especially for the minority 'No-show' class. The confusion matrix clarifies prediction types. Precision and Recall offer insights into the trade-off between false alarms and missed no-shows.

- **Accuracy:** The overall accuracy is **0.78**, this metric is heavily skewed by the model's strong performance on the majority 'Showed Up' class (Precision: 0.82, Recall: 0.93, F1-Score: 0.87).
- **Precision:** The critical 'No-show' class, the model exhibits a precision of **0.40**, meaning that only 40% of the times it predicted a no-show, it was correct.
- **Recall:** The recall is **0.19**, indicating it only captured 19% of the actual no-shows.
- **F1-Score:** The results in a low F1-score of **0.25** for the minority class, highlighting the model's significant challenge in effectively identifying no-show events.

Based on the classification report, the Random Forest model shows strong performance in predicting patients who will show up, as evidenced by high precision and recall for the 'Showed Up' class. However, its ability to accurately predict no-shows is limited, with a precision of 0.40, recall of 0.19, and an F1-score of 0.25.

6.2 Model 2: Logistic Regression

First, we run the code to evaluate the regression model. We are using a confusion matrix to evaluate the model and then calculate the ROC score and visualize the result in a curve graph.

```
[33]: print("--- Logistic Regression Model Evaluation ---")

# Classification Report
print("Classification Report:")
print(classification_report(y_test, y_pred_log_reg))

# Confusion Matrix
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred_log_reg))

# ROC AUC Score
roc_auc_log_reg = roc_auc_score(y_test, y_proba_log_reg)
print(f"\nROC AUC Score: {roc_auc_log_reg:.4f}")

# ROC Curve for Logistic Regression
fpr_log_reg, tpr_log_reg, _ = roc_curve(y_test, y_proba_log_reg)

plt.figure(figsize=(8, 6))
plt.plot(fpr_log_reg, tpr_log_reg, color='darkorange', lw=2, label=f'Logistic Regression (AUC = {roc_auc_log_reg:.2f})')
plt.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--', label='Random Classifier (AUC = 0.5)')
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver Operating Characteristic (ROC) Curve - Logistic Regression')
plt.legend(loc="lower right")
plt.grid(True)
plt.show()
```

The classification report provides a detailed breakdown of precision, recall, and F1-score for each class, along with overall accuracy.

```

--- Logistic Regression Model Evaluation ---
Classification Report:
              precision    recall  f1-score   support

     0       0.80        0.99        0.89    17642
     1       0.35        0.01        0.03     4464

 accuracy          0.80        22106
 macro avg       0.58        0.50        0.46    22106
 weighted avg    0.71        0.80        0.71    22106

```

Metrics for Class 0 (Show):

- Precision (0.80): When the model predicts a patient will show up, it is correct 80% of the time.
- Recall (0.99): The model correctly identifies 99% of all actual "show" cases. This is very high, meaning it's excellent at catching patients who will show up.
- F1-score (0.89): This is the harmonic mean of precision and recall. An F1-score of 0.89 indicates strong performance in classifying patients who show up.
- Support (17642): There are 17,642 actual "show" cases in the test set.

Metrics for Class 1 (No-show):

- Precision (0.35): When the model predicts a patient will *not* show up, it is correct only 35% of the time. This means there are many false positives – instances where the model predicts a no-show, but the patient actually shows up.
- Recall (0.01): This is very low. The model only correctly identifies 1% of all actual "no-show" cases. This is a significant issue, as it means the model misses almost all patients who are truly going to no-show.
- F1-score (0.03): The F1-score is extremely low, reflecting the poor balance between precision and recall for the "no-show" class.
- Support (4464): There are 4,464 actual "no-show" cases in the test set.

Overall Metrics:

- Accuracy (0.80): The model achieves an overall accuracy of 80%. While this number seems good in isolation, it is misleading due to the class imbalance. Since a large majority of patients (17,642 out of 22,106) show up, a model that simply predicts

everyone will show up would achieve very high accuracy, but would be useless for identifying no-shows.

- Macro Avg F1-score (0.46): This averages the F1-scores for both classes, treating them equally. It gives a more balanced view than overall accuracy when dealing with imbalanced datasets.
- Weighted Avg F1-score (0.71): This averages the F1-scores weighted by the number of instances in each class. It's closer to the overall accuracy because the "show" class dominates.

Next is the Confusion Matrix which gives a direct count of correct and incorrect predictions:

```
Confusion Matrix:  
[[17525  117]  
 [ 4400   64]]
```

- True Negatives (TN) = 17525: The model correctly predicted 17,525 patients would show up, and they indeed showed up.
- False Positives (FP) = 117: The model incorrectly predicted 117 patients would not show up, but they actually showed up. (Type I error)
- False Negatives (FN) = 4400: The model incorrectly predicted 4,400 patients would show up, but they actually did not show up. (Type II error). This is a very high number and directly correlates with the low recall for the 'no-show' class.
- True Positives (TP) = 64: The model correctly predicted only 64 patients would not show up, and they indeed did not show up.

The confusion matrix clearly highlights the model's primary weakness: it is very poor at identifying actual no-shows. It labels almost all no-shows as "show," leading to a high number of False Negatives.

To conclude, the Logistic Regression model, while achieving a decent overall accuracy of 80% (largely due to correctly classifying the majority class of "shows"), **struggles significantly with predicting "no-shows."** Its very low recall (1%) and precision (35%) for the "no-show" class, coupled with a high number of False Negatives (4400), mean that the model would be ineffective for its primary purpose of identifying high-risk patients for intervention. For practical application

in a healthcare setting to reduce no-shows, this model would likely need substantial improvement or a different approach to address the class imbalance and enhance its ability to correctly identify the minority "no-show" class.

7.0 Conclusion

Based on the evaluation metrics, both the Random Forest and Logistic Regression models demonstrate strong performance in predicting patients who are likely to show up. The overall accuracy from the random forest model is 0.78 while the overall accuracy from the logistic regression is 0.8 which means the logistic regression has higher accuracy. However, both models struggle significantly to predict actual “No-Show” patients, as they show low precision and recall values for this class.

This indicates that while the models are effective at identifying patients who will attend their appointments, they are much less reliable at detecting those who are at risk of missing them, which is the primary reason for this analysis.

To conclude, both models have shown the same problem which struggle to predict the actual “No-Show” patients. Therefore, we can say that the main cause of this problem comes from the dataset itself where the data for actual “No-Show” patients is lacking which results in this outcome. Thus, to improve this, future work can try balancing the data such adding more “No-Show” data or using better features that help the model recognize no-show patterns more clearly.

Project 2



UTM

UNIVERSITI TEKNOLOGI MALAYSIA

FACULTY OF COMPUTING

SEMESTER II

2024/2025

SECP 2753 – DATA MINING

SECTION 01

PROJECT 2 - UNSUPERVISED LEARNING

LECTURER: DR. ROLIANA BINTI IBRAHIM

NAME	MATRIC NUMBER
NURUL IKA SYAFINY BINTI AZHAR	A23CS0164
LUBNA AL HAANI BINTI RADZUAN	A23CS0107
NURAI SYAH BINTI MOHD ZIKRE	A23CS0160
NUR FIRZANA BINTI BADRUS HISHAM	A23CS0156

Table of Contents

1.0 INTRODUCTION	2
2.0 DATASET - No-Show Appointments	3
2.1 Data and Business Understanding	3
2.2 Data Preparation and Preprocessing	4
2.3 Model Development (Unsupervised Learning)	7
2.4 Model Performance & Evaluation (Elbow Method, Silhouette Score)	10
3.0 CONCLUSION	11

1.0 INTRODUCTION

In the contemporary healthcare landscape, scheduled appointments offer significant convenience for patients and optimize resource management within clinics and hospitals. However, a persistent challenge for healthcare providers is the issue of missed medical appointments, commonly known as "no-shows."

While some may view this as a minor inconvenience, it can lead to substantial waste of resources, impacting resource planning, staff efficiency, and potentially delaying care for other patients. To mitigate these negative effects, **unsupervised learning** offers a powerful approach. Instead of directly predicting individual no-shows, unsupervised methods can help in **discovering hidden patterns, segments, or characteristics within appointment data**. By clustering patients based on their historical behavior or identifying the underlying factors contributing to no-show patterns, healthcare systems can gain deeper insights, inform strategic interventions, and optimize resource allocation more effectively.

2.0 DATASET - No-Show Appointments

For this report, we are going to analyze the second dataset regarding patients who missed their medical appointments as well as running a machine learning model to predict whether the patient will likely show up to their medical appointment or not and evaluate its performance. The original dataset can be obtained from this link: [📄 noshowappointments](#)

2.1 Data and Business Understanding

Problem Statement:

To predict whether a patient will likely show up for their scheduled medical appointment.

Overview:

In this part, we will focus on understanding the business problem and exploring the dataset where the main goal is to predict whether a patient will likely show up for their scheduled medical appointment, to improve the management in the healthcare ecosystem. Firstly, we will explore the dataset structure, types of features, and initial data quality. Basic analysis helps identify key characteristics such as missing values, imbalanced classes, and anomalies (e.g., negative age). These insights guide the data cleaning and feature engineering steps that follow.

Importance:

- By identifying a patient that has high risk of missing their medical appointment, an alternative action can be taken by reminding them of their medical appointment and rescheduling it if any.
- Predicting a potential no-shows trends can help healthcare providers in managing the schedule by rescheduled the unused appointment to reduce downtime

2.2 Data Preparation and Preprocessing

For unsupervised learning, clustering, data preparation and preprocessing are equal. It is more important than supervised learning. This phase ensures that the input data is optimized for the clustering algorithm to identify the patterns and groupings without the guidance of predefined labels. It involves meticulous cleaning, feature engineering and most importantly the appropriate scaling of features.

The data preprocessing for Project 2 adapted some of the methods from Project 1, with key modifications tailored to unsupervised learning:

```
# 1. Initial Cleaning: Handle anomaly in Age
# Remove rows where Age is less than 0
initial_rows = df.shape[0]
df = df[df['Age'] >= 0]
rows_removed = initial_rows - df.shape[0]
print("Removed {} rows with invalid Age (Age < 0). Remaining rows: {}".format(rows_removed, df.shape[0]))
```

Removed 1 rows with invalid Age (Age < 0). Remaining rows: 110526

1. Handling Anomalies (Age = -1): The record containing an invalid Age value of -1 was removed to maintain the data integrity.

```
# 2. Feature Engineering from Date/Time Columns
df['ScheduledDay'] = pd.to_datetime(df['ScheduledDay'])
df['AppointmentDay'] = pd.to_datetime(df['AppointmentDay'])

# Ensure 'AppointmentDay' has no time component for accurate day difference calculation
df['AppointmentDay'] = df['AppointmentDay'].dt.normalize()

# Calculate 'WaitingDays': Difference between AppointmentDay and ScheduledDay
df['WaitingDays'] = (df['AppointmentDay'] - df['ScheduledDay']).dt.days
df.loc[df['WaitingDays'] < 0, 'WaitingDays'] = 0 # Cap negative waiting days at 0
print("Engineered 'WaitingDays' feature.")

# Extract Day of Week and Month
df['AppointmentDayOfWeek'] = df['AppointmentDay'].dt.dayofweek # Monday=0, Sunday=6
df['ScheduledDayOfWeek'] = df['ScheduledDay'].dt.dayofweek # Monday=0, Sunday=6
df['MonthOfAppointment'] = df['AppointmentDay'].dt.month
print("Engineered 'AppointmentDayOfWeek', 'ScheduledDayOfWeek', 'MonthOfAppointment'.")
```

Engineered 'WaitingDays' feature.
Engineered 'AppointmentDayOfWeek', 'ScheduledDayOfWeek', 'MonthOfAppointment'.

2. Feature Engineering: ScheduledDay and AppointmentDay columns were converted to datetime objects. The WaitingDays feature was engineered, calculating the duration between scheduling and appointment, with negative values capped at 0. Temporal features like AppointmentDayOfWeek (the day of the week on which the appointment is scheduled), ScheduledDayOfWeek (the day of the week when the appointment was scheduled) and MonthOfAppointment (the month in which the appointment falls) were extracted to capture cyclical patterns.

```

# 3. Categorical Feature Encoding
# Encode 'Gender': 'F' -> 0, 'M' -> 1
df['Gender'] = df['Gender'].map({'F': 0, 'M': 1})
print("Encoded 'Gender' column (F=0, M=1).")

# Prepare 'No-show' column but note it will be dropped later for unsupervised learning
# (Applying the same cleaning as in Project 1 to ensure data consistency if it were ever used)
df['No-show'] = df['No-show'].str.strip().str.capitalize()
df['No-show'] = df['No-show'].map({'No': 0, 'Yes': 1})
# Fill NaNs that might result from mapping (e.g., if other values existed)
if df['No-show'].isnull().any():
    print("WARNING: NaNs found in 'No-show' after mapping. Filling with 0. NaNs before fill: {}".format(df['No-show'].isnull().sum()))
    df['No-show'].fillna(0, inplace=True)
else:
    print("'No-show' column mapped successfully with no NaNs introduced (will be dropped).")

# Apply One-Hot Encoding to 'Neighbourhood'
# This handles the many unique values of Neighbourhood.
df_processed = pd.get_dummies(df, columns=['Neighbourhood'], prefix='Neighbourhood')
print("One-Hot Encoded 'Neighbourhood' column.")

Encoded 'Gender' column (F=0, M=1).
'No-show' column mapped successfully with no NaNs introduced (will be dropped).
One-Hot Encoded 'Neighbourhood' column.

```

3. Categorical Feature Encoding: The Gender column was numerically encoded (F to 0, M to 1). The Neighbourhood column, being a nominal categorical variable with numerous unique entries, was transformed using One-Hot Encoding. This process expands the single categorical column into multiple binary columns to prevent the algorithm from unintended ordinal relationship.

```

# 4. Feature Removal for Unsupervised Learning
# Drop PatientID, AppointmentID, original date columns (ScheduledDay, AppointmentDay)
# CRUCIAL: Also drop the 'No-show' column as it is the target in supervised learning,
# but for unsupervised clustering, we are looking for patterns in the features themselves.
columns_to_drop = ['PatientID', 'AppointmentID', 'ScheduledDay', 'AppointmentDay', 'No-show']
df_final_features = df_processed.drop(columns=columns_to_drop, axis=1)
print("Dropped identifier columns, original date columns, and the 'No-show' target variable.")

Dropped identifier columns, original date columns, and the 'No-show' target variable.

```

4. Feature Removal: Unique identifiers (PatientID, AppointmentID) and the original date columns were removed. In unsupervised learning, the goal is to discover inherent structures within the data itself, without relying on a predefined outcome.

```

# 5. Feature Scaling (CRUCIAL for K-Means Clustering)

numerical_cols_to_scale = ['Age', 'WaitingDays', 'AppointmentDayOfWeek', 'ScheduledDayOfWeek', 'MonthOfAppointment']

# Ensure only columns that exist are included in numerical_cols_to_scale
numerical_cols_to_scale = [col for col in numerical_cols_to_scale if col in df_final_features.columns]

if numerical_cols_to_scale:
    scaler = StandardScaler()
    df_final_features[numerical_cols_to_scale] = scaler.fit_transform(df_final_features[numerical_cols_to_scale])
    print("Applied StandardScaler to numerical features: {}".format(', '.join(numerical_cols_to_scale)))
else:
    print("No numerical columns found for scaling.")

```

5. Feature Scaling (Standardization): This step is very important for K-Means clustering. K-Means works by measuring distance between data points, so if one feature (like Age or WaitingDays) has much bigger numbers than others, it can affect the results too much and make the other features less important.

To fix this, we used StandardScaler on the numerical features. This method changes the values so that each feature has an average of 0 and the same spread, which is standard deviation of 1.

By doing this, all the features are on the same scale, so they each play an equal role in the clustering. This avoids one feature from having too much influence just because of its larger values.

	Gender	Age	Scholarship	Hipertension	Diabetes	Alcoholism	Handcap	SMS_received	WaitingDays	AppointmentDayOfWeek	...	Neighbourhood_SANTOS REIS	N
0	0	1.077926	0	1	0	0	0	0	-0.63435	1.561421	...	False	
1	1	0.818297	0	0	0	0	0	0	-0.63435	1.561421	...	False	
2	0	1.077926	0	0	0	0	0	0	-0.63435	1.561421	...	False	
3	0	-1.258733	0	0	0	0	0	0	-0.63435	1.561421	...	False	
4	0	0.818297	0	1	1	0	0	0	-0.63435	1.561421	...	False	

The preprocessing pipeline successfully transformed the raw data into a suitable format for unsupervised learning. The Age anomaly was handled, and various temporal features were engineered. Categorical variables like Gender and Neighbourhood were appropriately encoded. All numerical features identified for scaling have been standardized to ensure they contribute equally to the algorithms like K-Means. The final df_final_features DataFrame is now shown and ready for clustering. [df_final_noshows.csv](#)

2.3 Model Development (Unsupervised Learning)

Unsupervised machine learning model used in this project is the K-Means model. K-means is a clustering algorithm used to partition n observations into k clusters, where each observation belongs to the cluster with the nearest mean (centroid).

1. Prune irrelevant features from preprocessed data

The development process start with pruning irrelevant features from preprocessed data:

```
# Choosing appropriate feature for K-means machine learning from preprocessed data
# These are chosen to find clusters and the characteristics of those clusters

# Identify all neighbourhood columns
neighbourhood_cols = [col for col in df_final_features.columns if 'Neighbourhood_' in col]

# Create a new DataFrame without irrelevant columns for clustering
X = df_final_features.drop(columns=neighbourhood_cols)
X = X.drop(columns=['AppointmentDayOfWeek', 'cluster', 'cluster_label'], errors='ignore')
```

All columns that have 'Neighbourhood_' at the front and column named 'AppointmentDaysOfWeek' are pruned to reduce the dimensionality of the features that will be used for clustering. As a result, features that is left are 'Gender', 'Age', 'Scholarship', 'Hipertension', 'Diabetes', 'Alcoholism', 'Handcap', 'SMS_receieved', 'WaitingDays', 'ScheduledDayOfWeek', and 'MonthOfAppointment'.

Columns named 'cluster' and 'cluster_label' are dropped in case there are old columns named with those names from the previous run of the code.

2. Determining Optimal Number of Clusters (k)

Next, the Elbow Method and Silhouette Score are used to determine the optimal number of clusters for the dataset. For the Elbow Method, the range of k values tested is from 1 to 11, resulting in a total of 10 trials using a random state of 0. Meanwhile, the Silhouette Score uses a range of k values from 2 to 11, with a total of 9 trials, since the minimum number of clusters must be 2. The code snippet for both methods are as follows:

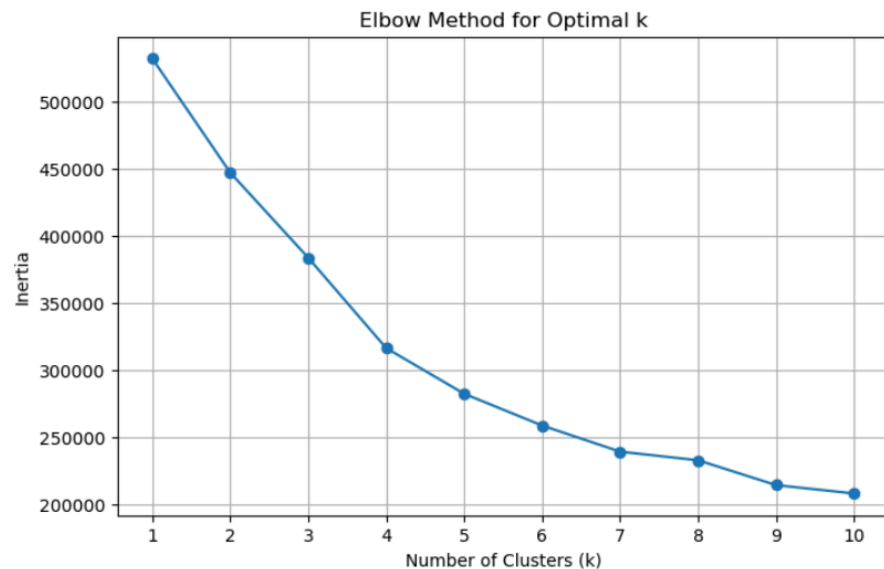
```
# Find Elbow point for k
inertia = []
for k in range(1, 11): # Test k from 1 to 10
    kmeans = KMeans(n_clusters=k, random_state=0, n_init='auto')
    kmeans.fit(X) # Use X (df_final_features) as your data input
    inertia.append(kmeans.inertia_)

plt.figure(figsize=(8, 5))
plt.plot(range(1, 11), inertia, marker='o')
plt.title('Elbow Method for Optimal k')
plt.xlabel('Number of Clusters (k)')
plt.ylabel('Inertia')
plt.xticks(range(1,11)) # Ensure x-axis ticks show all k values
plt.grid(True)
plt.show()
```

```
# Find Silhouette Scores for different k
silhouette_scores = []
for k in range(2, 11): # Silhouette score requires at least 2 clusters
    kmeans = KMeans(n_clusters=k, random_state=0, n_init='auto')
    cluster_labels = kmeans.fit_predict(X) # Use X (df_final_features)
    score = silhouette_score(X, cluster_labels)
    silhouette_scores.append(score)
    print(f"k={k}, Silhouette Score: {score:.3f}")

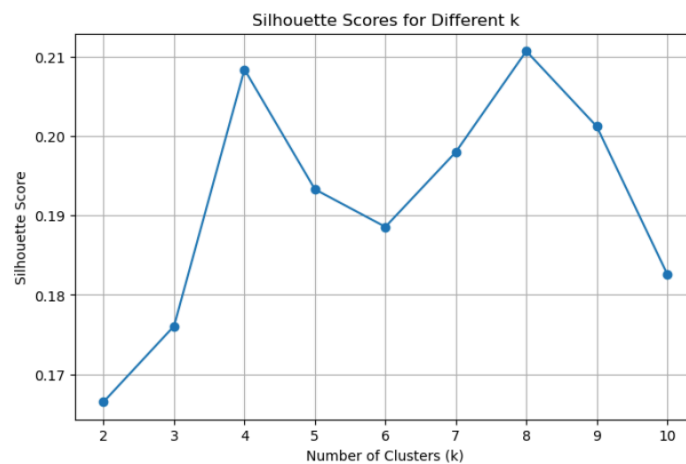
# plotting these scores for better visualization
plt.figure(figsize=(8, 5))
plt.plot(range(2, 11), silhouette_scores, marker='o')
plt.title('Silhouette Scores for Different k')
plt.xlabel('Number of Clusters (k)')
plt.ylabel('Silhouette Score')
plt.xticks(range(2,11))
plt.grid(True)
plt.show()
```

The result of both methods are visualized using plot graphs. From the elbow method, the result shows that when $k=4$, is the optimal value of k . This is because as k increases, inertia always decreases but after a certain point, the rate of decrease slows down, this is where the “elbow” appears where $k=4$.



As observed from the silhouette score, $k = 8$ produced the best clustering quality (score = 0.211). However, $k = 3$ or 4 also produced reasonably strong scores (0.176 and 0.208) and were easier to interpret. Thus, for this project, we selected $k = 4$ based on a balance of mathematical quality and practical usefulness.

k=2, Silhouette Score: 0.167
k=3, Silhouette Score: 0.176
k=4, Silhouette Score: 0.208
k=5, Silhouette Score: 0.193
k=6, Silhouette Score: 0.189
k=7, Silhouette Score: 0.198
k=8, Silhouette Score: 0.211
k=9, Silhouette Score: 0.201
k=10, Silhouette Score: 0.183



After selecting the optimal number of clusters ($k = 4$), the K-Means algorithm was applied to the standardized dataset using the selected features. Each patient record was assigned to one of the four clusters based on their similarity across the selected variables.

The characteristics of each cluster were then analyzed by calculating the mean values of the features within each group. This helped to interpret what each cluster represents in terms of patient profiles. The clusters revealed distinct groups of patients, for example:

- Cluster 0: Middle-aged patients with moderate health risks but long waiting times
- Cluster 1: Younger individuals with no chronic conditions and short waiting times
- Cluster 2: Elderly patients with multiple health issues such as hypertension and diabetes
- Cluster 3: Patients with scholarships and a higher likelihood of receiving SMS reminders

Silhouette Score for this clustering is: 0.274

	Gender	Age	Scholarship	Hipertension	Diabetes	Alcoholism	Handcap	SMS_received	WaitingDays	ScheduledDayOfWeek	MonthOfAppointment	cluster
cluster												
0	0.31	0.09	0.08	0.15	0.05	0.02	0.01	0.62	2.20	0.05	-0.27	0.0
1	0.39	-0.82	0.13	0.01	0.00	0.01	0.01	0.24	-0.28	-0.06	-0.52	1.0
2	0.32	0.90	0.07	0.42	0.16	0.06	0.03	0.25	-0.32	0.06	-0.52	2.0
3	0.35	0.01	0.10	0.20	0.07	0.03	0.02	0.40	-0.16	-0.02	1.67	3.0

These clusters provide meaningful groupings that can be used to better understand patient behaviors and risk patterns. For example, patients in high-risk clusters (Cluster 2) may benefit from targeted reminders or improved appointment scheduling to reduce the likelihood of missed appointments.

2.4 Model Performance & Evaluation (Elbow Method, Silhouette Score)

Since K-Means is an unsupervised algorithm, traditional classification evaluation metrics such as accuracy, precision, recall and F1-score are not applicable. Instead, clustering performance is evaluated using internal validation methods that assess the quality of the clusters based on how cohesive and distinctive the clusters are with each other. The following are the methods used:

1. Silhouette Score

The silhouette score measures how well each data point fits within its cluster compared to other clusters. It ranges from -1 to 1, where higher values indicate better-defined clusters. In this project, the silhouette score was calculated for each value of k from 2 to 10.

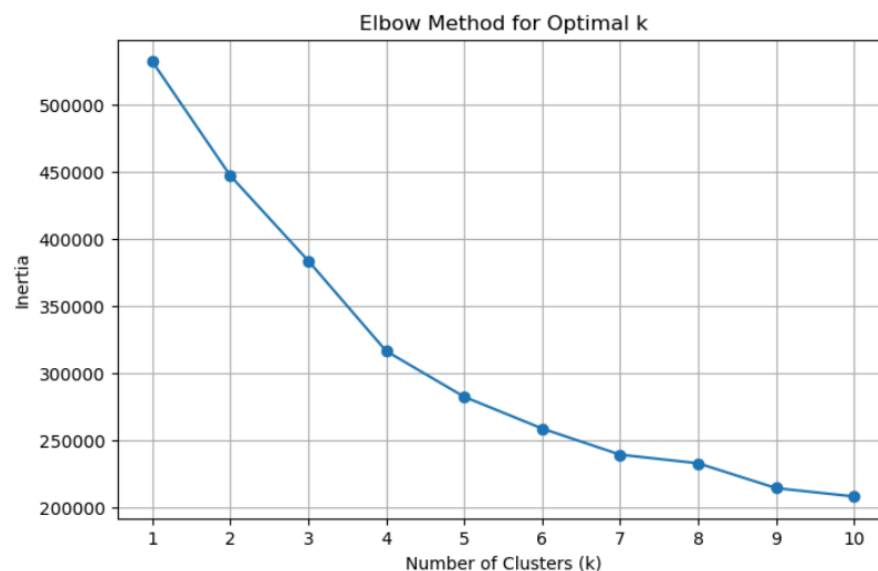
The highest silhouette score was observed at $k = 8$ (0.211), but $k = 4$ was selected due to better interpretability and sufficiently high score (0.208). However, after calculating the silhouette score again with a constant value of $k=4$ after cleaning the dataset again, the silhouette score results with a value of 0.274 which is an increase from 0.208.

Silhouette Score for this clustering is: 0.274

2. Elbow Method (Inertia)

Inertia is the sum of squared distances between each point and its assigned cluster center. A lower inertia indicates tighter clusters. However, since inertia always decreases as k increases, the elbow method helps identify the point where the rate of improvement sharply declines.

The elbow was observed at $k = 4$, indicating a good trade-off between model complexity and cluster cohesion.



3.0 CONCLUSION

This project successfully applied unsupervised machine learning techniques, specifically K-Means clustering, to uncover patterns in patient behavior regarding medical appointment attendance. After comprehensive data preparation and feature engineering, the Elbow Method and Silhouette Score were used to determine the optimal number of clusters. Although the silhouette score peaked at $k = 8$, $k = 4$ was selected based on a balance of interpretability, clustering quality, and practical usefulness.

Four different patient groups were identified by the final model, which had a silhouette score of 0.274 and was based on scheduling behavior, SMS reminders, health conditions, and demographics. These clusters can help healthcare providers identify high-risk patient profiles, such as elderly individuals with chronic conditions or patients with long waiting periods, who may require targeted actions.

By understanding the characteristics of each cluster, healthcare facilities can develop more effective appointment scheduling strategies and communication approaches to reduce no-show rates. This not only improves resource planning and efficiency but also enhances patient care delivery.

To further enhance clustering quality and actionability, future research could examine the use of supervised machine learning models to explicitly predict no-shows or take into account extra behavioral and social characteristics.